

中国科学技术大学

本科毕业论文



随机特征法求解高维线性抛物方程

作者姓名：	刘行
学 号：	PB22000150
专 业：	信息与计算科学
导 师：	陈景润 教授
完成时间：	2026 年 5 月 4 日

中国科学技术大学本科毕业论文学术诚信承诺书

本人郑重承诺：所呈交的本科毕业论文，是本人在导师指导下进行研究工作所取得的成果。除已特别加以标注和致谢的地方外，论文中不包含任何他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的贡献均已在论文中作了明确的说明。如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文(设计)等违纪行为。

作者签名：_____

签字日期：_____

摘要

高维偏微分方程 (Partial Differential Equations, PDEs) 在金融, 量子物理, 控制问题等领域都具有重要应用. 然而, 维度诅咒使得传统数值方法在高维情形下计算复杂度呈指数增长, 严重限制了其适用范围. 近年来, 基于机器学习的函数逼近方法为高维问题提供了新的思路. 随机特征法 (Random Feature Method, RFM) 是一类通过构造随机特征基函数对目标函数进行线性表示的方法, 其核心思想是利用随机特征映射将隐式定义的核函数转化为特征函数的数学期望, 并通过有限样本平均进行近似, 从而避免显式构造高维 Gram 矩阵, 将问题转化为低维最小二乘问题. 本文提出一种结合随机特征法与蒙特卡洛采样的数值方法, 用于高效求解高维线性与半线性抛物型 PDEs. 该方法利用随机采样处理高维空间积分, 同时借助随机特征表示降低函数逼近复杂度, 在计算效率与精度之间取得平衡. 首先将方程转化为随机微分方程 (Stochastic Differential Equations, SDEs), 并确定待求解的目标点. 然后从目标点出发进行蒙特卡洛采样, 并学习解函数在路径点上的梯度及目标点处的函数值. 其中梯度项采用随机特征映射进行逼近, 训练过程使用最小二乘法以提高计算效率. 数值实验包括线性的热方程, Black-Scholes 方程, 以及半线性的 HJB 方程和 Allen-Cahn 方程. 实验结果表明该方法具有较高效率.

关键词: 高维偏微分方程; 随机特征法; 蒙特卡洛方法; 随机微分方程; 最小二乘法

ABSTRACT

High-dimensional partial differential equations (PDEs) arise in a wide range of applications, including finance, quantum physics, and control theory. However, the curse of dimensionality causes the computational complexity of traditional numerical methods to grow exponentially with respect to the dimension, severely limiting their applicability. In recent years, machine learning-based function approximation methods have provided new perspectives for tackling high-dimensional problems. The Random Feature Method (RFM) represents a class of approaches that approximate target functions via linear combinations of randomly generated feature functions. Its key idea is to employ random feature mappings to transform implicitly defined kernel functions into expectations over feature functions, which are then approximated by finite sample averages. This avoids the explicit construction of high-dimensional Gram matrices and reduces the problem to a low-dimensional least-squares formulation. In this work, a numerical method combining the Random Feature Method with Monte Carlo sampling is proposed for efficiently solving high-dimensional linear and semilinear parabolic PDEs. The method leverages random sampling to handle high-dimensional integrals, while using random feature representations to reduce the complexity of function approximation, achieving a balance between computational efficiency and accuracy. The PDE is first reformulated as a system of stochastic differential equations (SDEs), and a target point is specified for evaluation. Monte Carlo sampling is then performed starting from the target point, and both the gradient of the solution along sampled paths and the function value at the target point are learned. The gradient term is approximated using random feature mappings, and the training process employs least-squares optimization to further improve efficiency. Numerical experiments on the linear heat equation, the Black–Scholes equation, with the semilinear HJB equation, and the Allen–Cahn equation demonstrate the effectiveness and efficiency of the proposed method.

Key Words: High-dimensional PDEs; Random feature method; Monte Carlo method; Stochastic differential equations; Least-squares method

目 录

第一章 引言	4
第一节 研究背景与意义	4
第二节 抛物型方程的基本类型	4
一、线性抛物型方程	4
二、半线性抛物型方程	6
第三节 随机特征方法	9
第四节 本文主要工作与章节安排	9
第二章 求解高维偏微分方程的随机特征方法	10
第一节 整体思路	10
第二节 样本路径与随机特征的构造	11
第三节 线性方程的求解	11
第四节 半线性方程的求解	13
第五节 测试阶段与误差评估	15
第三章 数值实验	16
第一节 实验设置	16
第二节 线性方程	16
第三节 半线性方程	17
第四节 参数敏感度分析	18
一、样本数的影响	18
二、随机特征宽度的影响	19
三、维度与运行时间	20
四、随机种子稳定性	21
第五节 与 DeepBSDE 方法的对比	22
第六节 本章小结	22
参考文献	24
附录 A 迭代格式推导	25
第一节 从 PDE 到 BSDE	25
第二节 线性方程终端格式的展开	25
第三节 半线性方程的灵敏度递推	26

附录 B 实验算法伪代码	28
第一节 统一实验框架	28
第二节 线性方程求解流程	29
第三节 半线性方程求解流程	30
第四节 测试误差评估流程	31
附录 C 参考值的计算与可靠性说明	32
第一节 Heat 方程的显式参考值	32
第二节 HJB-LQ 方程的积分参考值	32
第三节 Black-Scholes 方程的 Monte Carlo 参考值	33
第四节 Allen-Cahn 方程的有限差分参考值	34
致谢	36

插图和附表清单

图 2.1	实验求解方法的整体流程	11
图 3.1	半线性方程的 LM 迭代残差下降	18
图 3.2	样本数对泛化比值和初值相对误差的影响	19
图 3.3	随机特征宽度对测试残差和初值相对误差的影响	19
图 3.4	维度变化下的运行时间和初值相对误差	21
图 3.5	不同随机种子下的初值相对误差	21
图 3.6	RFM 与 DeepBSDE 的初值相对误差对比	22
表 3.1	主要实验配置	16
表 3.2	线性方程主要结果	17
表 3.3	半线性方程主要结果	17
表 3.4	半线性方程的 LM 迭代结果	18
表 3.5	样本数变化结果	18
表 3.6	维度变化的平均结果	20
表 3.7	随机种子稳定性结果	21
表 3.8	RFM 与 DeepBSDE 在半线性方程上的结果对比	22
表 C.1	Heat 方程参考值	32
表 C.2	HJB-LQ 方程参考值	33
表 C.3	Black-Scholes 方程 Monte Carlo 参考值	34
表 C.4	Allen-Cahn 方程有限差分参考值	35

第一章 引言

第一节 研究背景与意义

偏微分方程 (Partial Differential Equations, PDEs) 是处理各种物理, 金融问题的重要手段. 近年来计算机硬件和计算科学的发展使得低维的简单问题基本可以快速求解. 但是由于传统的有限元方法在解决高维度问题时网格数量呈指数增长, 时间复杂度随之爆炸, 出现了人们常说的维度诅咒 (curse of dimensionality) 现象, 因此需要找到更高效的方法求解高维 PDEs. 蒙特卡洛采样通过避免全空间的网格生成解决了时间复杂度问题, 而前馈神经网络则解决了在蒙特卡洛采样下编码边界条件与优化解函数的问题. 随着深度学习的发展, 已经有很多研究开始使用此方法求解高维 PDEs. 本文则提出了一种改进方法, 使用随机特征法代替神经网络, 实现降低维度和线性化的目的.

第二节 抛物型方程的基本类型

抛物型方程是一类重要的偏微分方程, 其典型形式为

$$\frac{\partial u}{\partial t} = \mathcal{L}u + f, \quad (1.1)$$

其中 $\mathcal{L}$ 为关于空间变量的二阶线性椭圆算子. 该类方程通常用于描述随时间演化的扩散过程, 例如热传导, 概率密度演化以及金融衍生品定价等问题. 为了方便与 BSDE 对齐, 我们将椭圆算子拆分得到如下形式:

$$\begin{aligned} \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, x) \nabla_x^2 u(t, x)) \\ + \langle \mu(t, x), \nabla u(t, x) \rangle \\ + f(t, x, u(t, x), \sigma^T(t, x) \nabla u(t, x)) = 0. \end{aligned} \quad (1.2)$$

一、线性抛物型方程

首先考虑其中的线性方程, 即 f 是 $u, \nabla u$ 的线性函数. 一方面, 从数学结构上看, 线性抛物型方程具有良好的正则性与稳定性, 其解通常可以通过解析表示 (如基本解或 Green 函数) 或概率表示 (如 Feynman-Kac 公式) 来刻画, 可以作为 baseline 用来判断方法的正确性并评估计算效率. 另一方面, 线性的方程也可以像半线性方程那样通过 Itô 公式变换为 BSDE, 使得此方法可以简单的推广到非线性情形. 下面列举了常见的线性抛物型方程:

1. 热方程是最经典的线性抛物型偏微分方程之一, 最初来源于热传导问题, 也广泛用于描述粒子扩散, 浓度传播以及随机扰动在空间中的扩散过程. 从概

率角度看, 热方程与布朗运动具有直接联系, 其解可以表示为布朗运动终端函数的期望. 因此, 热方程不仅是扩散模型的基础方程, 也是检验随机方法求解线性抛物型 PDE 的标准基准问题. 在高维情形下, 热方程的维度对应空间变量的维数, 即 $x \in \mathbb{R}^d$ 中的 d . 在本文的统一半线性抛物型方程框架下, 热方程对应的随机过程为标准 d 维布朗运动, 即漂移项为零, 扩散系数为单位矩阵. 因此有

$$\sigma(t, x) = I_d, \quad \mu(t, x) = 0, \quad f(t, x, u, z) = 0.$$

该算例主要用于验证随机特征法在纯扩散线性问题上的基本逼近能力.

2. **Black–Scholes** 方程来源于金融衍生品定价理论, 是数学金融中最具有代表性的线性抛物型方程之一. Black 和 Scholes^[1] 在期权定价研究中将期权价格视为标的资产价格与时间的函数, 并通过无套利思想和动态对冲方法推导出相应的定价方程; 在经典单资产情形下, 该模型给出了欧式期权的封闭形式定价公式, 即把期权价值写成股票价格和时间的函数, 并通过构造对冲组合得到定价微分方程. 在多资产情形下, 若期权收益依赖于多个标的资产, 则 PDE 的空间维度等于资产数量, 因而 Black–Scholes 方程自然成为高维 PDE 方法中的典型应用算例. 设 $x = (x_1, \dots, x_d)$ 表示 d 个资产价格, r 表示无风险利率, Σ 表示资产收益率的协方差矩阵. 在风险中性测度下, 资产价格通常服从几何布朗运动, 其漂移项为 rx , 扩散项与当前资产价格成正比. 因此, 在本文的统一框架下, 多资产 Black–Scholes 方程对应

$$\sigma(t, x) = \text{diag}(x) \Sigma^{1/2}, \quad \mu(t, x) = rx, \quad f(t, x, u, z) = -ru.$$

其中 $-ru$ 项表示未来收益在无风险利率下的折现效应. 该算例用于检验本文方法在具有乘性扩散, 非零漂移和折现项的高维金融定价问题上的表现.

3. **Ornstein–Uhlenbeck** 方程对应的随机过程是一类典型的均值回复过程, 最早常用于描述带阻尼的随机运动, 后来也广泛出现在统计物理, 金融数学和随机控制等领域. 与布朗运动不同, Ornstein–Uhlenbeck 过程的漂移项会将状态变量拉回长期均值附近, 因而适合描述具有稳定中心或长期均衡水平的随机系统. 在高维情形下, 其维度同样对应状态变量的空间维数.
4. **线性反应–扩散**方程用于描述扩散过程与局部反应过程同时存在的现象. 其中扩散项刻画物质, 能量或信息在空间中的传播, 反应项刻画局部增长, 衰减或外部源项作用. 这类方程在化学反应, 生物种群动力学, 热源扩散以及物理场演化问题中都有广泛应用. 在本文中考虑的线性反应–扩散方程仍然采用标准扩散过程作为其随机过程部分, 即漂移项为零, 扩散系数为单位矩阵. 与热方程不同的是, 方程中额外加入了关于解函数 u 的线性反应项以及可能依赖于时间和空间的外部源项 $c(t, x)$.

尽管部分方程在物理建模中通常定义于低维空间 (如三维), 但在金融建模, 多因子随机过程等问题中, 其状态变量维度可以显著增加, 因此常被用作高维数值算法的基准问题. 由于前两者已经能包含大部分线性方程的形式, 且后面还有非线性方程的实验, 本论文将仅对热方程和 Black-Scholes 方程进行数值实验.

二、半线性抛物型方程

如果 f 不是 u 与 ∇u 的线性函数, 则整个方程为半线性抛物型方程. 尽管半线性方程解的解析表示不那么明显, 但通常也有较为完善的理论对其数值做近似估计. 在完成线性方程求解的理论分析与实验验证后, 对半线性方程的实验可以进一步说明此方法的有效性. 作为线性的推广, 我们依然通过 Itô 公式将此类方程变换为 BSDE 进行求解. 下面列举了常见的半线性抛物型方程:

1. **Hamilton-Jacobi-Bellman** 方程来源于动态规划和随机最优控制理论. 在最优控制问题中, 解函数通常表示从当前状态出发所能达到的最优代价, 因而也称为值函数. 当状态变量维数较高时, HJB 方程的维度等于控制系统的状态空间维度, 传统网格方法会迅速受到维数灾难的限制. 因此, HJB 方程是检验高维 PDE 方法的重要非线性算例. 本文采用线性二次型高斯控制问题对应的 HJB 方程. 设状态变量 $X_t \in \mathbb{R}^d$ 满足受控随机微分方程

$$dX_t = 2\sqrt{\lambda} m_t dt + \sqrt{2} dW_t,$$

其中 m_t 为控制变量, $\lambda > 0$ 表示控制强度, W_t 为 d 维标准布朗运动. 控制目标为最小化代价泛函

$$J(\{m_t\}_{0 \leq t \leq T}) = \mathbb{E} \left[\int_0^T \|m_t\|^2 dt + g(X_T) \right].$$

由动态规划原理可得到对应的 HJB 方程

$$\frac{\partial u}{\partial t}(t, x) + \Delta u(t, x) - \lambda \|\nabla_x u(t, x)\|^2 = 0, \quad u(T, x) = g(x).$$

在本文的统一半线性抛物型方程框架下, 取

$$\mu(t, x) = 0, \quad \sigma(t, x) = \sqrt{2} I_d.$$

此时

$$z = \sigma^T \nabla_x u = \sqrt{2} \nabla_x u,$$

因而非线性项可以写为

$$f(t, x, y, z) = -\frac{\lambda}{2} \|z\|^2.$$

实验中可取

$$d = 100, \quad T = 1, \quad x_0 = 0, \quad \lambda = 1,$$

终端条件取

$$g(x) = \log \left(\frac{1 + \|x\|^2}{2} \right).$$

对于该例, 可利用指数变换得到参考表达式

$$u(t, x) = -\frac{1}{\lambda} \log \mathbb{E} \left[\exp \left(-\lambda g(x + \sqrt{2} W_{T-t}) \right) \right],$$

因而可以通过高精度 Monte Carlo 方法计算参考值. 该算例主要用于检验本文方法处理梯度型非线性项的能力.

2. Allen–Cahn 方程是一类典型的非线性反应–扩散方程, 常用于描述相分离, 界面演化以及有序–无序转变等物理现象. 其核心特点是同时包含扩散项和双势阱型反应项. 扩散项刻画空间传播或平滑效应, 非线性反应项刻画系统向不同稳定相的演化趋势. 与 HJB 方程不同, Allen–Cahn 方程的非线性主要依赖于解函数 u 本身, 而不是梯度. 一个常用的 Allen–Cahn 方程形式为

$$\frac{\partial v}{\partial s}(s, x) = \Delta v(s, x) + v(s, x) - v^3(s, x), \quad v(0, x) = g(x).$$

为了与本文采用的终端值问题形式一致, 令

$$u(t, x) = v(T - t, x),$$

则方程可改写为

$$\frac{\partial u}{\partial t}(t, x) + \Delta u(t, x) + u(t, x) - u^3(t, x) = 0, \quad u(T, x) = g(x).$$

在统一半线性抛物型方程框架下, 取

$$\mu(t, x) = 0, \quad \sigma(t, x) = \sqrt{2} I_d,$$

并令

$$f(t, x, y, z) = y - y^3.$$

实验中可取

$$d = 100, \quad T = 0.3, \quad x_0 = 0,$$

终端条件取

$$g(x) = \frac{1}{2 + 0.4\|x\|^2}.$$

本文求解的目标量为

$$u(0, 0),$$

它对应原始正向时间方程中 $v(T, 0)$ 的值. 由于该方程一般没有简单闭式解, 可采用文献中的高精度数值结果或大规模 Monte Carlo 类方法作为参考. 该算例主要用于检验本文方法对反应型非线性项的处理能力.

3. 带违约风险的非线性 Black–Scholes 方程是金融衍生品定价中的一个重要扩展模型. 经典 Black–Scholes 方程假设市场不存在违约风险, 此时方程是线性的. 但在实际金融市场中, 合约发行方可能发生违约, 合约持有人只能获得部分价值补偿. 若违约强度依赖于当前合约价值, 则定价方程中会出现关于 u 的非线性项. 设 $x = (x_1, \dots, x_d)$ 表示 d 个标的资产价格, 在风险中性测度下资产价格可建模为几何布朗运动. 一类典型的带违约风险 Black–Scholes 方程可写为

$$\frac{\partial u}{\partial t}(t, x) + \bar{\mu}x \cdot \nabla_x u(t, x) + \frac{\bar{\sigma}^2}{2} \sum_{i=1}^d x_i^2 \frac{\partial^2 u}{\partial x_i^2}(t, x) - (1 - \delta)Q(u(t, x))u(t, x) - Ru(t, x) = 0.$$

其中 R 为无风险利率, $\delta \in [0, 1)$ 表示违约后的回收比例, $Q(u)$ 表示与当前合约价值有关的违约强度. 在统一框架下, 该方程可对应

$$\mu(t, x) = \bar{\mu}x, \quad \sigma(t, x) = \bar{\sigma} \operatorname{diag}(x),$$

以及

$$f(t, x, y, z) = -(1 - \delta)Q(y)y - Ry.$$

该方程展示了金融定价模型中非线性项的一个典型来源: 市场风险因素或信用风险因素会使得原本线性的定价方程变为非线性.

4. Fisher–KPP 方程是另一类经典的非线性反应–扩散方程, 最初用于描述种群扩散和有利基因传播等问题. 其典型特点是反应项具有 Logistic 增长结构: 当解较小时近似线性增长, 当解接近饱和值时增长受到抑制. 因此, 该方程常被用来刻画带有扩散机制的传播前沿问题. 一个典型形式为

$$\frac{\partial v}{\partial s}(s, x) = \Delta v(s, x) + v(s, x)(1 - v(s, x)).$$

通过同样的时间反向变换 $u(t, x) = v(T - t, x)$, 可以写成终端值问题

$$\frac{\partial u}{\partial t}(t, x) + \Delta u(t, x) + u(t, x)(1 - u(t, x)) = 0.$$

在统一框架下, 可取

$$\mu(t, x) = 0, \quad \sigma(t, x) = \sqrt{2}I_d,$$

以及

$$f(t, x, y, z) = y(1 - y).$$

与 Allen–Cahn 方程类似, Fisher–KPP 方程也是反应–扩散型方程, 但其非线性项对应的是 Logistic 型增长, 而不是双势阱结构.

第三节 随机特征方法

通常的神经网络通过多层线性变换与非线性激活函数的叠加来逼近目标函数,这会导致需要求解一个非线性的非凸优化问题.随机特征法的核心思想是将内层参数随机初始化并固定,仅对输出层参数进行优化,从而将原问题转化为一个凸优化问题,在平方损失函数下进一步退化为最小二乘问题.这种方法通过减少需要优化的参数自由度来显著简化训练过程,以牺牲一定的表达能力为代价换取更高的计算效率与稳定性.由于内层参数固定,随机特征法本质上可以视为对核方法的一种近似,即通过随机特征映射将核函数表示为特征函数的数学期望,并利用有限样本平均进行近似.

在实际应用中,随机特征的分布及尺度参数会显著影响逼近能力.通过合理选择这些参数可以使特征函数张成更丰富的函数空间,从而提高数值求解的精度.因此,其在保持较低优化复杂度的同时,为高维函数逼近提供了一种有效的替代方案.特别地,由于其线性结构,随机特征法在基于最小二乘的偏微分方程数值方法中具有天然优势.

第四节 本文主要工作与章节安排

第二章 求解高维偏微分方程的随机特征方法

考虑方程 (1.2) 的终值问题, 即

$$\begin{aligned} \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, x) \nabla_x^2 u(t, x)) + \langle \mu(t, x), \nabla u(t, x) \rangle \\ + f(t, x, u(t, x), \sigma^T(t, x) \nabla u(t, x)) = 0, \quad (t, x) \in [0, T) \times \mathbb{R}^d, \quad (2.1) \\ u(T, x) = g(x), \quad x \in \mathbb{R}^d. \end{aligned}$$

为了应用 Itô 公式, 假设 $u \in C^{1,2}([0, T) \times \mathbb{R}^d)$, 并要求 μ, σ, f, g 满足足够的正则性与增长条件. 令状态过程满足

$$X_t = \xi + \int_0^t \mu(s, X_s) ds + \int_0^t \sigma(s, X_s) dW_s, \quad (2.2)$$

再定义

$$Y_t := u(t, X_t), \quad Z_t := \sigma^T(t, X_t) \nabla u(t, X_t). \quad (2.3)$$

则原方程可写成 BSDE

$$dY_t = -f(t, X_t, Y_t, Z_t) dt + Z_t^T dW_t, \quad (2.4)$$

并满足终端条件 $Y_T = g(X_T)$. 从 (2.1) 到 (2.4) 的推导见附录 A 第一节.

取时间剖分 $0 = t_0 < t_1 < \dots < t_N = T$, 其中 $\Delta t = T/N$, 并记 $\Delta W_k = W_{t_{k+1}} - W_{t_k}$. 对 (2.4) 采用 Euler 离散后, 得到

$$Y_{k+1} = Y_k - \Delta t f(t_k, X_k, Y_k, Z_k) + \langle Z_k, \Delta W_k \rangle. \quad (2.5)$$

因而问题转化为: 在离散路径上同时近似初值 Y_0 和各时间层的梯度项 Z_k , 使终端值与 $g(X_T)$ 尽可能一致.

第一节 整体思路

本文的实验流程可以概括为四个阶段:

1. 根据给定方程生成离散样本路径;
2. 在所有时间层和路径点上构造随机特征, 用于近似 Z_k ;
3. 根据方程类型选择线性最小二乘或非线性 Levenberg-Marquardt 方法求解参数;
4. 在独立测试路径上重新推进终端值, 计算测试残差.

上述四步中, 前两步构成公共的数据准备阶段, 对线性和半线性问题都相同; 第三步则是两类方程的主要差异所在. 在线性情形下, 末端匹配可以直接化为一个最小二乘系统; 在半线性情形下, 则需要通过迭代方式不断修正参数. 算法的流程图如图 2.1 给出. 对应的较详细伪代码见附录 B.

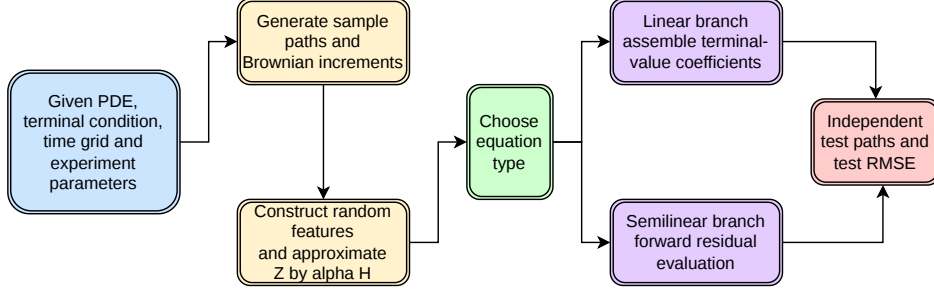


图 2.1 实验求解方法的整体流程

第二节 样本路径与随机特征的构造

设终值问题已经被离散为 N 个时间步, 样本数为 S . 对每个实验算例, 首先按照对应随机微分方程生成 S 条独立样本路径

$$\{X_k^{(i)}\}_{k=0}^N, \quad i = 1, 2, \dots, S,$$

以及与之对应的布朗增量

$$\{\Delta W_k^{(i)}\}_{k=0}^{N-1}.$$

这些路径同时承担两个作用: 一方面它们给出离散 BSDE 的推进轨道, 另一方面它们也是随机特征映射的输入样本.

在得到路径后, 对每个时间层和路径点计算随机特征

$$H_k^{(i)} = \tanh(Ax_k^{(i)} + bt_k + c),$$

其中 A, b, c 为固定随机参数. 本文始终只训练输出层参数 α , 并用

$$Z_k^{(i)} \approx \alpha H_k^{(i)}$$

作为梯度项的统一近似. 于是原问题的求解就转化为对 y_0 和 α 的求解.

第三节 线性方程的求解

对线性方程, 由第二章的推导可知, 每条路径的终端值都可以写成

$$y_N^{(i)} = \beta_i y_0 + \langle \Gamma_i, \alpha \rangle + \eta_i.$$

因而所有路径拼接后, 就得到一个关于参数向量 $(y_0, \text{vec}(\alpha))$ 的线性最小二乘问题.

从实验流程的角度看, 线性情形可分为以下几步:

1. 生成样本路径并计算随机特征;
2. 根据离散递推式计算每条路径的系数 $\beta_i, \Gamma_i, \eta_i$;
3. 将所有路径系数拼接成统一的设计矩阵;
4. 求解最小二乘问题得到 y_0 与 α ;

5. 用求得的参数回代, 计算训练样本上的终端残差.

线性流程的特点在于参数只需求解一次, 不需要额外的外层迭代. 因此实验中线性部分主要用于观察样本数, 特征宽度以及维度变化对最小二乘近似效果的影响.

先考虑线性生成元

$$f(t, \mathbf{x}, y, \mathbf{z}) = \mathcal{L}(t, \mathbf{x})y + \langle \mathcal{M}(t, \mathbf{x}), \mathbf{z} \rangle + \mathcal{N}(t, \mathbf{x}). \quad (2.6)$$

其中 $\langle \mathcal{M}, \mathbf{z} \rangle$ 可以并入漂移项处理, 因此后文只考虑 $\mathcal{M} \equiv 0$ 的情形.

对每条样本路径 i , 用随机特征映射逼近梯度项

$$\mathbf{Z}_k^{(i)} \approx \alpha \mathbf{H}_k^{(i)}, \quad \mathbf{H}_k^{(i)} = \tanh(\mathbf{A}x_k^{(i)} + bt_k + c), \quad (2.7)$$

其中 $\mathbf{A}, b, c$ 为固定随机参数, α 为待训练的输出层参数. 代入 (2.5) 后得到离散迭代

$$y_{k+1}^{(i)} = (1 - \Delta t \mathcal{L}(t_k, x_k^{(i)}))y_k^{(i)} + (\Delta \mathbf{W}_k^{(i)})^T \alpha \mathbf{H}_k^{(i)} - \Delta t \mathcal{N}(t_k, x_k^{(i)}). \quad (2.8)$$

由于上式关于 y_0 和 α 是线性的, 终端值 $y_N^{(i)}$ 也必然是它们的仿射函数. 将递推完全展开后, 可写为

$$y_N^{(i)} = \beta_i y_0 + \langle \Gamma_i, \alpha \rangle + \eta_i, \quad (2.9)$$

其中 $\beta_i, \Gamma_i, \eta_i$ 由路径 $\{x_k^{(i)}, \Delta \mathbf{W}_k^{(i)}\}$ 唯一确定, 具体表达式分别见附录 A 第二节的 (A.12), (A.13) 和 (A.14).

将所有路径的终端值拼接后, 便得到线性最小二乘问题. 记

$$\boldsymbol{\vartheta} = \begin{bmatrix} y_0 \\ \text{vec}(\alpha) \end{bmatrix}, \quad \mathbf{A}_{i,\cdot} = \begin{bmatrix} \beta_i & \text{vec}(\Gamma_i)^T \end{bmatrix}, \quad \mathbf{B}_i = g(x_N^{(i)}) - \eta_i. \quad (2.10)$$

则线性方程最终求解

$$\min_{\boldsymbol{\vartheta}} \|\mathbf{A}\boldsymbol{\vartheta} - \mathbf{B}\|_2^2 + \lambda_r \|\mathbf{R}\boldsymbol{\vartheta}\|_2^2, \quad (2.11)$$

其中 $\lambda_r > 0$ 为 ridge 正则化参数,

$$\mathbf{R} = \begin{bmatrix} 0 & 0 \\ 0 & \mathbf{I}_{dH} \end{bmatrix}. \quad (2.12)$$

也就是说, 正则化只作用在随机特征输出层参数 α 上, 不惩罚初值 y_0 . 这样做可以避免把 y_0 人为向零偏移, 同时抑制病态线性系统给出的过大 α .

实际计算时, 设未知量个数为 $p = 1 + dH$. 当样本数 $S \geq p$ 时, 直接求解 primal ridge 系统

$$(\mathbf{A}^T \mathbf{A} + \lambda_r \mathbf{R}^T \mathbf{R}) \boldsymbol{\vartheta} = \mathbf{A}^T \mathbf{B}. \quad (2.13)$$

当 $S < p$ 时, 使用对偶形式降低线性系统规模. 由于当前实现的对偶形式主要用于欠定情形, 其正则化写为

$$\vartheta = A^T (AA^T + \lambda_r I_S)^{-1} B. \quad (2.14)$$

线性情形的优势在于不需要迭代更新随机特征参数, 每次实验只需求解一个最小二乘系统即可.

第四节 半线性方程的求解

对半线性方程, 终端值关于参数不再保持线性, 因而不能直接写成单个最小二乘系统. 这时需要把参数记为

$$\theta = (y_0, \text{vec}(\alpha)),$$

并通过最小化终端残差平方和来求解

$$\min_{\theta} \frac{1}{2} \|r(\theta)\|_2^2.$$

本文实验中对半线性方程采用 Levenberg–Marquardt 方法. 每次迭代都围绕当前参数执行以下流程:

1. 根据当前参数构造 $Z_k^{(i)} = \alpha H_k^{(i)}$;
2. 沿所有样本路径前向推进离散 BSDE, 得到终端残差;
3. 计算残差关于参数的雅可比矩阵;
4. 求解阻尼线性子问题, 得到参数增量;
5. 比较更新前后的目标函数值, 决定是否接受该步并调整阻尼参数.

这一步骤会持续进行, 直到终端残差足够小, 参数更新足够小, 或达到最大迭代次数. 因而半线性实验的核心并不在于单次前向传播本身, 而在于“前向残差评估 + 雅可比构造 + 阻尼更新”这一循环是否稳定.

对一般半线性生成元 $f(t, \mathbf{x}, y, \mathbf{z})$, 仍采用相同的随机特征近似

$$Z_k^{(i)} \approx \alpha H_k^{(i)}. \quad (2.15)$$

此时离散迭代式为

$$y_{k+1}^{(i)} = y_k^{(i)} - f(t_k, x_k^{(i)}, y_k^{(i)}, \alpha H_k^{(i)}) \Delta t + (\Delta W_k^{(i)})^T \alpha H_k^{(i)}. \quad (2.16)$$

由于生成元 f 关于 y 或 z 不再保持线性, 终端值 $y_N^{(i)}$ 不能再写成关于参数的线性函数, 因此末端匹配需要改写为半线性最小二乘问题.

令参数向量为

$$\theta = \begin{bmatrix} y_0 \\ \text{vec}(\alpha) \end{bmatrix}, \quad (2.17)$$

对第 i 条路径定义终端残差

$$r_i(\theta) = y_N^{(i)}(\theta) - g(x_N^{(i)}), \quad (2.18)$$

并记

$$r(\theta) = \begin{bmatrix} r_1(\theta) \\ \vdots \\ r_S(\theta) \end{bmatrix}. \quad (2.19)$$

最终求解问题为

$$\min_{\theta} \frac{1}{2} \|r(\theta)\|_2^2. \quad (2.20)$$

本文采用 Levenberg–Marquardt 方法求解 (2.20). 在第 m 次迭代中, 将残差在当前参数 $\theta^{(m)}$ 处线性化:

$$r(\theta^{(m)} + \delta) \approx r(\theta^{(m)}) + J(\theta^{(m)})\delta, \quad (2.21)$$

其中

$$J(\theta^{(m)}) = \frac{\partial r}{\partial \theta}(\theta^{(m)}) \quad (2.22)$$

为雅可比矩阵. 于是每步需要求解阻尼线性系统

$$(J^T J + \lambda I)\delta^{(m)} = -J^T r, \quad \lambda > 0, \quad (2.23)$$

并更新

$$\theta^{(m+1)} = \theta^{(m)} + \delta^{(m)}. \quad (2.24)$$

若目标函数下降, 则接受该步并减小阻尼参数; 否则拒绝该步并增大阻尼参数.

为了构造 (2.23), 需要计算终端残差对参数的灵敏度. 对第 i 条路径定义

$$S_k^{(i)} = \frac{\partial y_k^{(i)}}{\partial \theta}, \quad (2.25)$$

则有初值

$$S_0^{(i)} = \begin{bmatrix} 1 & 0 & \cdots & 0 \end{bmatrix}, \quad (2.26)$$

并满足递推

$$S_{k+1}^{(i)} = \left(1 - \Delta t f_{y,k}^{(i)}\right) S_k^{(i)} + \left((\Delta W_k^{(i)})^T - \Delta t f_{z,k}^{(i)}\right) \begin{bmatrix} 0_{d \times 1} & I_d \otimes (H_k^{(i)})^T \end{bmatrix}, \quad (2.27)$$

其中

$$f_{y,k}^{(i)} = \frac{\partial f}{\partial y}(t_k, x_k^{(i)}, y_k^{(i)}, \alpha H_k^{(i)}), \quad f_{z,k}^{(i)} = \frac{\partial f}{\partial z}(t_k, x_k^{(i)}, y_k^{(i)}, \alpha H_k^{(i)}). \quad (2.28)$$

因而第 i 条路径对应的雅可比行向量就是

$$J_i(\theta) = S_N^{(i)} = \left(\prod_{j=0}^{N-1} A_j^{(i)}\right) S_0^{(i)} + \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} A_j^{(i)}\right) B_k^{(i)}. \quad (2.29)$$

上述灵敏度递推及其由来见附录 A 第三节.

这与线性方程中系数矩阵的推导形式类似, 区别在于此处 $A_k^{(i)}$ 和 $B_k^{(i)}$ 依赖当前迭代步中的 $y_k^{(i)}$ 和 α , 因而需要在每一次迭代中重新计算. 但随机特征结构使待优化参数始终只出现在输出层, 从而显著降低了优化问题的规模.

第二章已经给出了线性方程与半线性方程对应的离散化模型和优化问题. 为了便于后续实验分析, 本章从算法执行流程的角度总结本文方法, 重点说明样本路径生成, 随机特征构造, 参数求解与测试评估之间的关系.

第五节 测试阶段与误差评估

为了避免只在固定训练路径上评价模型, 本文在训练完成后还会重新采样独立测试路径. 对测试路径仍然使用相同的随机特征参数和输出层参数, 并按照与训练阶段一致的离散格式推进终端值. 由此得到的测试误差记为

$$\text{RMSE}_{\text{test}} = \sqrt{\frac{1}{S} \sum_{i=1}^S (Y_N^{(i)} - g(X_N^{(i)}))^2}.$$

若训练误差下降而测试误差没有同步下降, 则说明随机特征近似可能更偏向于拟合当前训练路径; 反之, 若两者同时下降, 则说明该参数设置具有更稳定的泛化效果. 因而训练 RMSE 与测试 RMSE 的对比是本实验章节中的核心观察指标之一.

第三章 数值实验

本章通过两个线性方程和两个半线性方程测试前文方法. 线性方程选取 Black–Scholes 方程和热方程, 主要观察线性最小二乘求解器的精度; 半线性方程选取 HJB-LQ 方程和 Allen–Cahn 方程, 用于观察 Levenberg–Marquardt 方法在高维 BSDE 离散系统中的效果.

第一节 实验设置

每个算例都先生成样本路径 $\{X_k^{(i)}, \Delta W_k^{(i)}\}_{k=0}^{N-1}$, 然后用随机特征函数近似各时间层的 Z_k . 对线性方程, 终端值 Y_N 关于 y_0 和输出层参数 α 保持线性, 所以可以直接求解带 ridge 正则的最小二乘问题. 对半线性方程, 终端残差关于参数不再是线性的, 本文采用 LM 方法求解相应的非线性最小二乘问题.

本章统一报告训练样本上的终端残差 RMSE 和独立测试样本上的终端残差 RMSE. 前者反映模型在固定样本路径上的拟合程度, 后者用于观察参数是否只适应了训练路径. 除特别说明外, 标准配置见表 3.1.

表 3.1 主要实验配置

方程	类型	d	T	N	S	H	λ_0	求解器
Black–Scholes	线性	100	1.0	20	16384	50	0.010	线性最小二乘
Heat	线性	100	1.0	20	16384	50	0.010	线性最小二乘
HJB-LQ	半线性	100	1.0	20	16384	50	0.010	LM
Allen–Cahn	半线性	100	0.3	20	16384	50	0.010	LM

本章还大量使用了方程初值的参考真实值, 其具体计算方法和误差或置信区间均见附录 C.

第二节 线性方程

本节考虑生成元关于 (y, z) 为线性的方程. 对这类问题, 离散 BSDE 的终端值可以写成

$$Y_N^{(i)} = A_{i,0}y_0 + \sum_{j=1}^{dH} A_{i,j}\theta_j + b_i, \quad \theta = \text{vec}(\alpha). \quad (3.1)$$

因此, 由终端匹配条件 $Y_N^{(i)} \approx g(X_N^{(i)})$ 可以直接得到一个线性最小二乘问题, 不需要再做外层非线性迭代.

Black-Scholes 算例采用多资产算术平均看涨期权. 状态过程满足

$$X_{k+1}^m = X_k^m \exp\left((r - \frac{1}{2}\sigma^2)\Delta t + \sigma\Delta W_k^m\right), \quad m = 1, \dots, d, \quad (3.2)$$

生成元和终端条件为

$$f(t, x, y, z) = -ry, \quad g(x) = \max\left(\frac{1}{d} \sum_{m=1}^d x_m - K, 0\right). \quad (3.3)$$

Heat 算例采用

$$X_t = x_0 + W_t, \quad f(t, x, y, z) = 0, \quad g(x) = \exp\left(-\frac{\|x\|^2}{a}\right). \quad (3.4)$$

表 3.2 线性方程主要结果

方程	y_0	参考值	相对误差	训练 RMSE	测试 RMSE
Black-Scholes	0.048726	0.048813	0.178%	0.0529	0.0746
Heat	0.372581	0.371528	0.283%	0.0360	0.0517

表 3.2 给出了两个线性算例的结果. 可以看到, 两个算例得到的初值都与独立参考值接近. Black-Scholes 的相对误差约为 0.18%, Heat 方程的相对误差约为 0.28%.

第三节 半线性方程

HJB-LQ 方程的非线性来自梯度项, Allen-Cahn 方程的非线性来自反应项. 这两个算例分别对应 z 型非线性和 y 型非线性.

HJB-LQ 方程采用

$$X_t = \sqrt{2}W_t, \quad f(t, x, y, z) = -\frac{1}{2}\|z\|^2, \quad g(x) = \log\left(\frac{1 + \|x\|^2}{2}\right). \quad (3.5)$$

Allen-Cahn 方程采用

$$X_t = \sqrt{2}W_t, \quad f(t, x, y, z) = y - y^3, \quad g(x) = \frac{1}{2 + 0.4\|x\|^2}. \quad (3.6)$$

表 3.3 半线性方程主要结果

方程	y_0	参考值	相对误差	训练 RMSE	测试 RMSE
HJB-LQ	4.590000	4.590162	0.0035%	0.0990	0.1443
Allen-Cahn	0.053131	0.052785	0.656%	0.0035	0.0051

表 3.3 表明, LM 求解后的两个半线性算例也能给出较准确的初值. HJB-LQ 的相对误差约为 0.0035%, Allen-Cahn 的相对误差约为 0.66%.

表 3.4 半线性方程的 LM 迭代结果

方程	迭代步数	接受步数	最终 λ	最终步长范数	时间 / ms
HJB-LQ	4	3	1.00×10^4	2.82×10^{-7}	3377.930
Allen-Cahn	4	3	1.00×10^6	5.59×10^{-10}	3148.710

图 3.1 给出了半线性求解器的残差下降过程. 横轴为 LM 接受步编号, 纵轴为训练终端残差 RMSE.

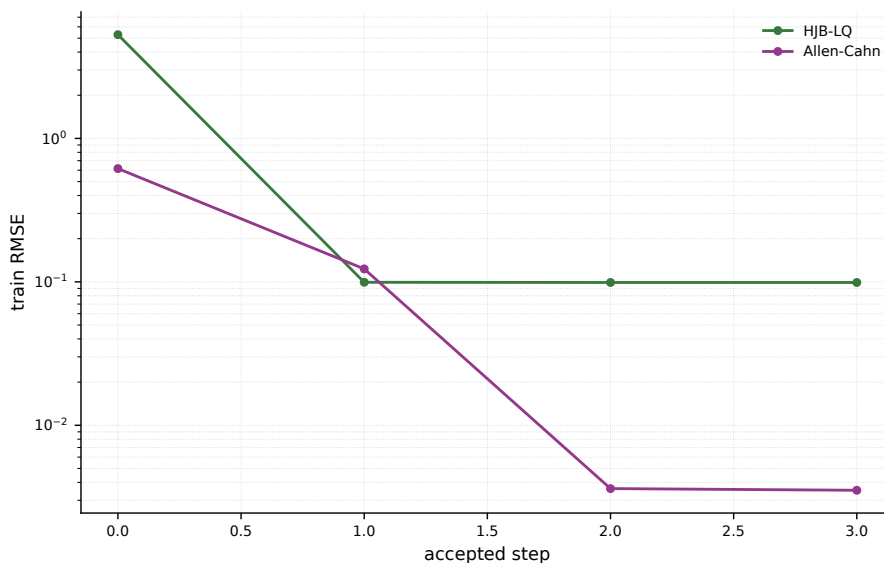


图 3.1 半线性方程的 LM 迭代残差下降

第四节 参数敏感度分析

一、样本数的影响

样本数会影响训练路径对真实分布的代表性. 表 3.5 固定 $d = 100$, $H = 50$, 只改变训练样本数 S .

表 3.5 样本数变化结果

方程	S	y_0	相对误差	训练 RMSE	测试 RMSE	测试/训练
Heat	4096	0.376310	1.287%	0.0002	0.1414	846.37
Heat	8192	0.373278	0.471%	0.0270	0.0696	2.57
Heat	16384	0.372581	0.283%	0.0360	0.0517	1.44
Black-Scholes	8192	0.048520	0.599%	0.0084	0.0199	2.38
Black-Scholes	16384	0.048726	0.178%	0.0529	0.0746	1.41
HJB-LQ	4096	4.423910	3.622%	1.36×10^{-6}	0.4595	3.37×10^5
HJB-LQ	8192	4.578700	0.250%	0.0762	0.1963	2.58
HJB-LQ	16384	4.590000	0.0035%	0.0990	0.1443	1.46
Allen-Cahn	8192	0.053268	0.916%	0.0028	0.0069	2.49
Allen-Cahn	16384	0.053131	0.656%	0.0035	0.0051	1.45

当样本数较小时, 训练残差本身不一定可靠. 随机种子决定了内层随机特征, 也会影响有效特征的数量; 在样本较少时, 某些随机特征组合可能很好地拟合训练路径, 但在独立测试路径上误差仍然较大. 因此表中加入测试 RMSE 与训练 RMSE 的比值, 作为观察泛化情况的辅助指标. 例如 HJB-LQ 在 $S = 4096$ 时训练 RMSE 几乎为零, 但测试/训练比值达到 3.37×10^5 , 初值相对误差也偏大. 因此后续分析主要参考独立测试残差和初值相对误差. 图 3.2 给出了测试/训练比值和初值相对误差的对比.

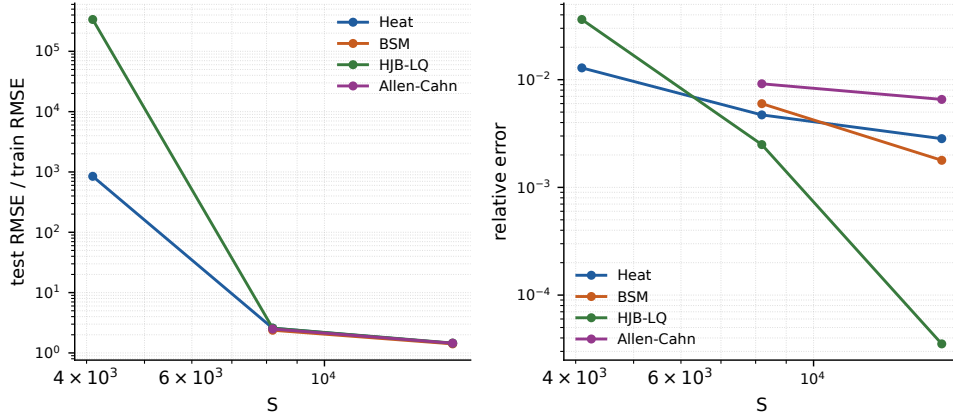


图 3.2 样本数对泛化比值和初值相对误差的影响

二、随机特征宽度的影响

随机特征宽度 H 决定每个时间层上可训练输出参数的数量. 本节固定 $d = 100$, $S = 16384$, 只改变 H . 对 HJB-LQ 方程, 增大 H 后初值相对误差和训练残差都有改善; 对 Heat, Black-Scholes 和 Allen-Cahn, 当 H 已达到当前规模后, 继续增大 H 并不能稳定带来更高精度. 这说明随机特征宽度并不是唯一的限制因素; 样本路径数量, 随机特征的有效线性无关程度以及最小二乘系统的条件数都会共同影响结果. 图 3.3 展示了测试残差和初值相对误差随 H 变化的情况.

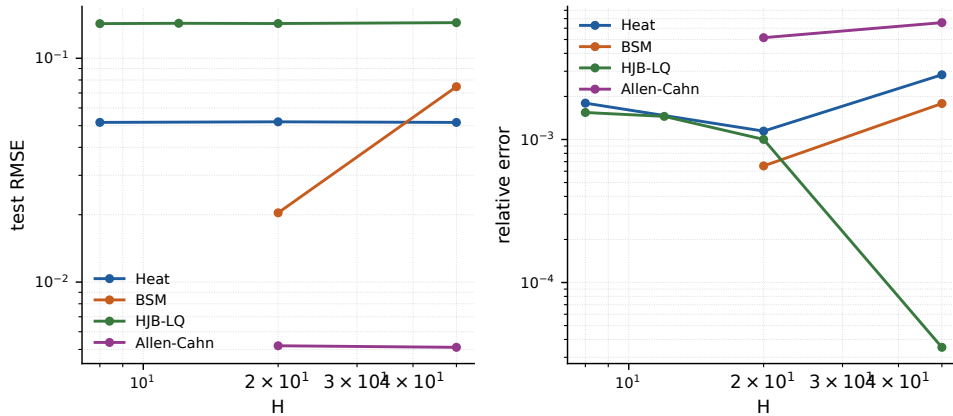


图 3.3 随机特征宽度对测试残差和初值相对误差的影响

三、维度与运行时间

为了观察维度变化对运行时间的影响, 本节固定 $H = 50$, $S = 16384$, 对 $d = 20, 30, 50, 100, 200, 300$ 分别运行三个随机种子并取平均. 由于总运行时间包含初始化, 公用数据构造和求解等多个部分, 直接用总时间拟合维度阶数并不合适. 因此对相邻两组维度 d_{j-1} 和 d_j , 定义局部阶

$$p_j = \frac{\log(t_j/t_{j-1})}{\log(d_j/d_{j-1})}, \quad (3.7)$$

其中 t_j 表示该维度下三次运行的平均时间. 表 3.6 列出了不同维度下的平均时间和相邻维度的局部阶.

表 3.6 维度变化的平均结果

方程	d	时间 / ms	p_j	方程	d	时间 / ms	p_j
Heat	20	204.4 ± 3.5	—	Black-Scholes	20	209.5 ± 5.1	—
Heat	30	216.0 ± 3.6	0.14	Black-Scholes	30	219.1 ± 2.0	0.11
Heat	50	238.8 ± 2.7	0.20	Black-Scholes	50	242.7 ± 4.2	0.20
Heat	100	314.7 ± 3.8	0.40	Black-Scholes	100	325.3 ± 4.5	0.42
Heat	200	568.8 ± 1.3	0.85	Black-Scholes	200	599.1 ± 9.9	0.88
Heat	300	1031.7 ± 4.8	1.47	Black-Scholes	300	1057.3 ± 0.3	1.40
HJB-LQ	20	435.5 ± 69.2	—	Allen-Cahn	20	444.7 ± 65.6	—
HJB-LQ	30	680.6 ± 92.5	1.10	Allen-Cahn	30	648.0 ± 112.1	0.93
HJB-LQ	50	1053.4 ± 176.5	0.85	Allen-Cahn	50	856.4 ± 90.0	0.55
HJB-LQ	100	3273.2 ± 263.6	1.64	Allen-Cahn	100	2780.4 ± 649.2	1.70
HJB-LQ	200	5616.8 ± 3150.0	0.78	Allen-Cahn	200	6472.6 ± 3419.0	1.22
HJB-LQ	300	27300.8 ± 4323.0	3.90	Allen-Cahn	300	40998.0 ± 7671.0	4.55

从表 3.6 可以看到, 线性方程在低维到中等维度上的局部阶较小, 到 $d = 200, 300$ 时局部阶逐渐增大; 半线性方程的时间增长更快, 尤其在 $d = 300$ 时受到非线性迭代和线性代数规模的共同影响. 这里的高维端局部阶升高, 不宜直接解释为算法本身的理论复杂度. 一方面, 维度增大后, 参数矩阵和样本路径占用的显存增加较多, GPU 并行度可能受到显存带宽, 缓存命中率和临时张量分配的限制; 另一方面, 半线性方程每次 LM 试探步都要重新传播路径并构造灵敏度, 拒绝步和重试次数会直接放大运行时间. 此外, 当问题接近实验设备的显存上限时, 内存碎片, kernel 调度开销, 以及矩阵求解例程在大尺寸矩阵上的工作区需求都可能造成额外开销. 本实验中 $d = 300$, $H = 50$, $S = 16384$ 已接近设备可运行上限, 因而这个端点更适合作为硬件压力测试, 而不是复杂度阶数的稳定估计点. 图 3.4 左图在 log-log 坐标下展示了平均运行时间随维度变化的趋势. 除运行时间外, 还需要检查高维下的初值精度. 图 3.4 右图给出了不同维度, 不同随机种子下的初值相对误差. 图中的误差大多仍然较小, 说明方法在较高维度下仍能给出有效初值.

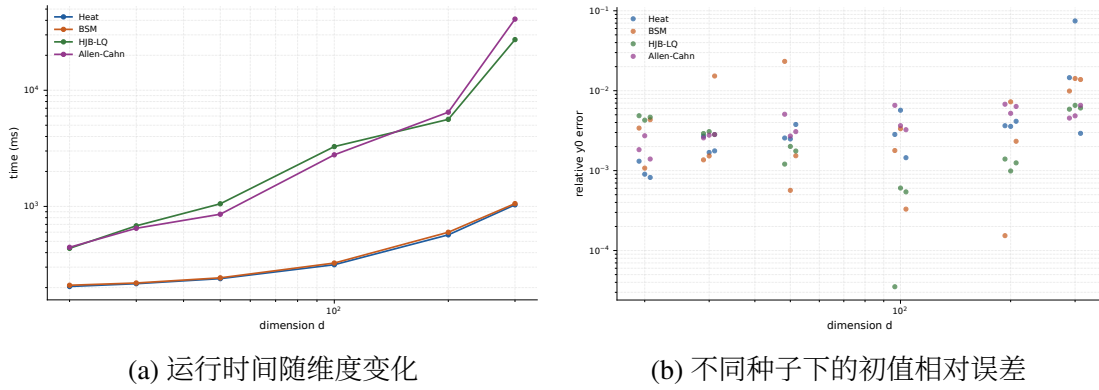


图 3.4 维度变化下的运行时间和初值相对误差

四、随机种子稳定性

随机特征内层参数和样本路径都由随机数生成, 因此需要检查结果是否依赖某个特定种子. 表 3.7 使用标准配置下的三个种子, 报告 y_0 均值, 样本标准差和平均测试残差.

表 3.7 随机种子稳定性结果

方程	y_0 均值	y_0 标准差	均值相对误差	测试 RMSE 均值
Heat	0.372764	8.04×10^{-4}	0.333%	0.0520
Black-Scholes	0.048843	1.26×10^{-4}	0.063%	0.0379
HJB-LQ	4.591863	1.62×10^{-3}	0.037%	0.1436
Allen-Cahn	0.053021	9.54×10^{-5}	0.449%	0.0051

四个方程的 y_0 波动都小于主要数值量级. 图 3.5 给出每个随机种子下的初值相对误差.

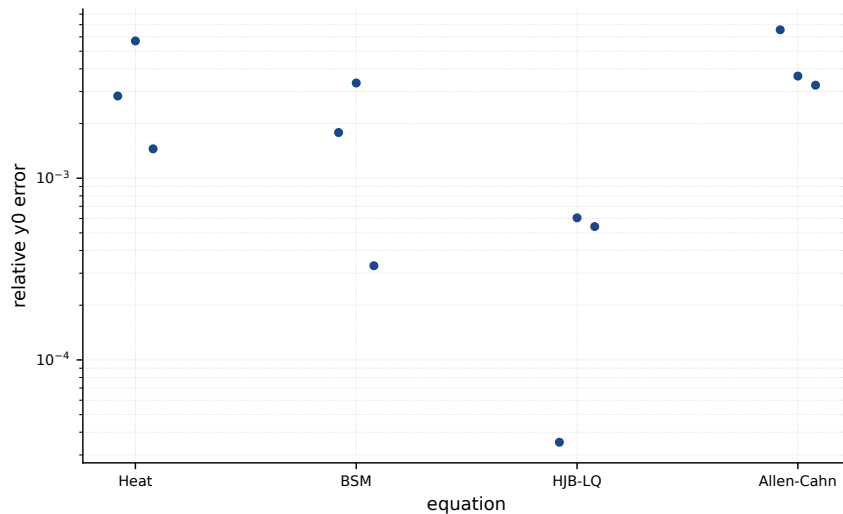


图 3.5 不同随机种子下的初值相对误差

第五节 与 DeepBSDE 方法的对比

Han 等^[2]提出了一种基于神经网络求解高维抛物型方程的方法, 后文暂称为 Deep BSDE. 该方法同样先将方程转化为 BSDE, 再拟合其中的梯度项. 不同的是, Deep BSDE 使用标准前馈神经网络 MLP 进行逼近, 训练过程相对更复杂. 为了进行比较, 本文实现了文献中的算法, 并在相同方程参数和相同环境下进行实验. 本文在 HJB-LQ 和 Allen-Cahn 两个半线性方程上记录了 DeepBSDE 的训练过程. 由于 DeepBSDE 的训练损失和本文 RFM 的终端残差并不是同一个优化目标, 因此这里不直接比较损失数值, 只比较初值相对误差随时间的变化.

表 3.8 RFM 与 DeepBSDE 在半线性方程上的结果对比

方程	方法	时间 / s	y_0	参考值	相对误差
HJB-LQ	RFM	3.378	4.590000	4.590162	0.0035%
HJB-LQ	DeepBSDE	15.221	4.586627	4.590162	0.077%
Allen-Cahn	RFM	3.149	0.053131	0.052785	0.656%
Allen-Cahn	DeepBSDE	29.192	0.052846	0.052785	0.116%

在 HJB-LQ 方程上, RFM 在更短时间内得到更小的初值相对误差. 在 Allen-Cahn 方程上, DeepBSDE 经过更长时间训练后初值相对误差略小, 而 RFM 的运行时间更短. 图 3.6 展示了初值相对误差随时间变化的过程.

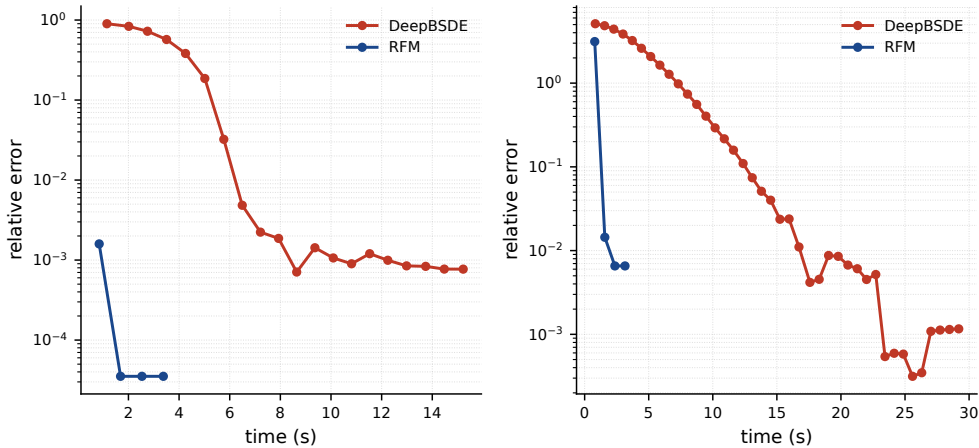


图 3.6 RFM 与 DeepBSDE 的初值相对误差对比

第六节 本章小结

本章的四个算例覆盖了线性最小二乘和半线性 LM 两种求解情形. 在线性方程中, Black-Scholes 与 Heat 方程的初值相对误差均低于 0.3%. 在半线性方程中, HJB-LQ 和 Allen-Cahn 也能在标准配置下得到接近参考值的初值. 样本数实验说

明, 训练残差过小并不一定代表泛化良好, 因而独立测试残差是必要的观察指标. 维度实验显示, 当前实现在线性方程上的时间增长较慢, 半线性方程的运行时间随维度增长更快; 同时, 高维下仍需要同时观察相对误差和测试残差, 例如 **Heat** 方程在 $d = 300$ 时出现了较明显的精度退化. 与 **DeepBSDE** 的对比表明, 在本文两个半线性算例上, 随机特征方法可以用较少训练时间得到接近参考值的初值.

参 考 文 献

- [1] Scholes M, Black F. The Pricing of Options and Corporate Liabilities | Journal of Political Economy: Vol 81, No 3[J]. Journal of Political Economy.
- [2] Han J, Jentzen A, E W. Solving high-dimensional partial differential equations using deep learning[J]. Proceedings of the National Academy of Sciences, 2018, 115(34): 8505-8510.

附录 A 迭代格式推导

本章给出第二章中若干关键公式的详细推导.

第一节 从 PDE 到 BSDE

考虑状态过程 (2.2) 和光滑函数 $u(t, x)$. 由 Itô 公式,

$$\begin{aligned} du(t, X_t) &= \frac{\partial u}{\partial t}(t, X_t) dt + \nabla u(t, X_t)^T dX_t \\ &\quad + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, X_t) \nabla_x^2 u(t, X_t)) dt. \end{aligned} \quad (\text{A.1})$$

将 (2.2) 代入上式, 得

$$\begin{aligned} du(t, X_t) &= \left(\frac{\partial u}{\partial t}(t, X_t) + \mu(t, X_t) \cdot \nabla u(t, X_t) \right. \\ &\quad \left. + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, X_t) \nabla_x^2 u(t, X_t)) \right) dt \\ &\quad + \nabla u(t, X_t)^T \sigma(t, X_t) dW_t. \end{aligned} \quad (\text{A.2})$$

若 u 满足方程 (1.2), 则

$$\begin{aligned} \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, x) \nabla_x^2 u(t, x)) \\ + \langle \mu(t, x), \nabla u(t, x) \rangle + f(t, x, u(t, x), \sigma^T(t, x) \nabla u(t, x)) = 0. \end{aligned}$$

代回 (A.2) 得

$$\begin{aligned} du(t, X_t) &= -f(t, X_t, u(t, X_t), \sigma^T(t, X_t) \nabla u(t, X_t)) dt \\ &\quad + \nabla u(t, X_t)^T \sigma(t, X_t) dW_t. \end{aligned} \quad (\text{A.3})$$

积分形式为

$$\begin{aligned} u(t, X_t) - u(0, X_0) &= - \int_0^t f(s, X_s, u(s, X_s), \sigma^T(s, X_s) \nabla u(s, X_s)) ds \\ &\quad + \int_0^t \nabla u(s, X_s)^T \sigma(s, X_s) dW_s. \end{aligned} \quad (\text{A.4})$$

再利用 (2.3) 的定义, 就得到正文中的 BSDE (2.4).

第二节 线性方程终端格式的展开

对线性生成元, 离散迭代式 (2.8) 可写为

$$y_{k+1}^{(i)} = a_k^{(i)} y_k^{(i)} + (\xi_k^{(i)})^T \alpha H_k^{(i)} + c_k^{(i)}, \quad (\text{A.5})$$

其中

$$a_k^{(i)} = 1 - \Delta t \mathcal{L}(t_k, x_k^{(i)}), \quad (\text{A.6})$$

$$\xi_k^{(i)} = \Delta W_k^{(i)}, \quad (\text{A.7})$$

$$c_k^{(i)} = -\Delta t \mathcal{N}(t_k, x_k^{(i)}). \quad (\text{A.8})$$

从 $y_0^{(i)} = y_0$ 出发, 前几步展开为

$$y_1^{(i)} = a_0^{(i)} y_0 + (\xi_0^{(i)})^T \alpha H_0^{(i)} + c_0^{(i)}, \quad (\text{A.9})$$

$$\begin{aligned} y_2^{(i)} &= a_1^{(i)} y_1^{(i)} + (\xi_1^{(i)})^T \alpha H_1^{(i)} + c_1^{(i)} \\ &= a_1^{(i)} a_0^{(i)} y_0 + a_1^{(i)} (\xi_0^{(i)})^T \alpha H_0^{(i)} + (\xi_1^{(i)})^T \alpha H_1^{(i)} \\ &\quad + a_1^{(i)} c_0^{(i)} + c_1^{(i)}. \end{aligned} \quad (\text{A.10})$$

由此可见, 每一步都会把已有项乘上新的系数 $a_k^{(i)}$, 再加上当前时刻的随机特征项和常数项. 归纳可得

$$\begin{aligned} y_N^{(i)} &= \left(\prod_{j=0}^{N-1} a_j^{(i)} \right) y_0 \\ &\quad + \left\langle \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} a_j^{(i)} \right) \xi_k^{(i)} (H_k^{(i)})^T, \alpha \right\rangle \\ &\quad + \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} a_j^{(i)} \right) c_k^{(i)}. \end{aligned} \quad (\text{A.11})$$

这里约定当 $k = N - 1$ 时, 空乘积取为 1.

将 (A.11) 与正文中的 (2.9) 对照, 可直接识别

$$\beta_i = \prod_{j=0}^{N-1} a_j^{(i)}, \quad (\text{A.12})$$

$$\Gamma_i = \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} a_j^{(i)} \right) \xi_k^{(i)} (H_k^{(i)})^T, \quad (\text{A.13})$$

$$\eta_i = \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} a_j^{(i)} \right) c_k^{(i)}. \quad (\text{A.14})$$

因而线性情形确实归结为关于 y_0 和 α 的线性最小二乘问题.

第三节 半线性方程的灵敏度递推

对半线性情形, 令

$$z_k^{(i)} = \alpha H_k^{(i)}.$$

由于参数向量取为 (2.17), 故

$$\frac{\partial z_k^{(i)}}{\partial \theta} = \begin{bmatrix} 0_{d \times 1} & I_d \otimes (H_k^{(i)})^T \end{bmatrix}. \quad (\text{A.15})$$

记

$$S_k^{(i)} = \frac{\partial y_k^{(i)}}{\partial \theta}.$$

因为 y_0 是参数向量的第一项, 而其余参数在初始时刻不直接进入 y_0 , 所以有初值 (2.26) .

对离散迭代式 (2.16) 关于 θ 求导:

$$\begin{aligned} S_{k+1}^{(i)} &= S_k^{(i)} - \Delta t \frac{\partial}{\partial \theta} f(t_k, x_k^{(i)}, y_k^{(i)}, z_k^{(i)}) \\ &\quad + (\Delta W_k^{(i)})^T \frac{\partial z_k^{(i)}}{\partial \theta}. \end{aligned}$$

对生成元使用链式法则,

$$\frac{\partial}{\partial \theta} f(t_k, x_k^{(i)}, y_k^{(i)}, z_k^{(i)}) = f_{y,k}^{(i)} S_k^{(i)} + f_{z,k}^{(i)} \frac{\partial z_k^{(i)}}{\partial \theta},$$

其中 $f_{y,k}^{(i)}$ 和 $f_{z,k}^{(i)}$ 的定义见 (2.28) . 代回上式得

$$\begin{aligned} S_{k+1}^{(i)} &= \left(1 - \Delta t f_{y,k}^{(i)}\right) S_k^{(i)} \\ &\quad + \left((\Delta W_k^{(i)})^T - \Delta t f_{z,k}^{(i)}\right) \frac{\partial z_k^{(i)}}{\partial \theta}. \end{aligned} \quad (\text{A.16})$$

再代入 (A.15) , 即得到正文中的递推式 (2.27) . 记

$$A_k^{(i)} = 1 - \Delta t f_{y,k}^{(i)}, \quad (\text{A.17})$$

$$B_k^{(i)} = ((\Delta W_k^{(i)})^T - \Delta t f_{z,k}^{(i)}) \begin{bmatrix} 0_{d \times 1} & I_d \otimes (H_k^{(i)})^T \end{bmatrix}. \quad (\text{A.18})$$

则

$$S_{k+1}^{(i)} = A_k^{(i)} S_k^{(i)} + B_k^{(i)}. \quad (\text{A.19})$$

展开可得

$$S_N^{(i)} = \left(\prod_{j=0}^{N-1} A_j^{(i)}\right) S_0^{(i)} + \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} A_j^{(i)}\right) B_k^{(i)}. \quad (\text{A.20})$$

由于

$$r_i(\theta) = y_N^{(i)}(\theta) - g(x_N^{(i)}),$$

而终端状态 $x_N^{(i)}$ 与参数 θ 无关, 故

$$\frac{\partial r_i}{\partial \theta} = \frac{\partial y_N^{(i)}}{\partial \theta} = S_N^{(i)}.$$

这就说明了雅可比矩阵可以由所有路径的终端灵敏度直接拼接得到.

附录 B 实验算法伪代码

本附录给出本文实验中实际采用的求解伪代码. 为了便于与正文中的方法分析对应, 这里不再强调具体实现细节, 而是统一使用“路径采样, 特征构造, 参数求解, 测试评估”的算法表述.

第一节 统一实验框架

算法 B.1 统一实验框架

Data: 方程参数, 时间剖分数 N , 样本数 S , 随机特征宽度 H

Result: 训练得到的 y_0, α , 以及训练与测试误差

- 1 生成训练样本路径 $\{X_k^{(i)}, \Delta W_k^{(i)}\}$;
 - 2 构造训练阶段的随机特征 $\{H_k^{(i)}\}$;
 - 3 **if** 方程为线性 **then**
 - 4 | 调用算法 B.2 求解 y_0 和 α ;
 - 5 **else**
 - 6 | 调用算法 B.3 求解 y_0 和 α ;
 - 7 **end**
 - 8 在训练路径上计算终端残差 RMSE;
 - 9 在独立测试路径上调用算法 B.4 计算测试 RMSE;
 - 10 返回求解参数与误差指标;
-

第二节 线性方程求解流程

算法 B.2 线性方程的求解流程

Data: 训练路径, 布朗增量, 随机特征, 线性方程离散系数, ridge 参数 λ_r

Result: y_0, α

- 1 对每条路径计算终端表达式中的系数 $\beta_i, \Gamma_i, \eta_i$;
- 2 将所有路径拼接为设计矩阵 A 与观测向量 B ;
- 3 令参数向量

$$\vartheta = \begin{bmatrix} y_0 \\ \text{vec}(\alpha) \end{bmatrix},$$

并记 $p = 1 + dH$;

- 4 构造

$$R = \begin{bmatrix} 0 & 0 \\ 0 & I_{dH} \end{bmatrix}$$

用于仅对 α 正则化;

- 5 **if** $S \geq p$ **then**

- 6 求解 primal ridge 系统

$$(A^\top A + \lambda_r R^\top R) \vartheta = A^\top B$$

得到参数 θ ;

- 7 **else**

- 8 求解对偶 ridge 系统

$$\vartheta = A^\top (AA^\top + \lambda_r I_S)^{-1} B$$

得到参数 θ ;

- 9 **end**

- 10 解包 θ 得到参数 y_0 与 α ;

- 11 返回 y_0 与 α ;

第三节 半线性方程求解流程

算法 B.3 半线性方程的 Levenberg–Marquardt 求解流程

Data: 训练路径, 布朗增量, 随机特征, 初始参数 $\theta^{(0)}$, 初始阻尼参数 $\lambda^{(0)}$

Result: y_0, α

```

1  令  $m \leftarrow 0$ ;
2  while 未满足停止准则 do
3      根据当前参数  $\theta^{(m)}$  构造所有  $Z_k^{(i)}$ ;
4      前向推进离散 BSDE, 计算终端残差向量  $r(\theta^{(m)})$ ;
5      计算雅可比矩阵  $J(\theta^{(m)})$ ;
6      求解阻尼子问题
          
$$(J^\top J + \lambda^{(m)} I) \delta^{(m)} = -J^\top r;$$

          令试探参数  $\theta_{\text{trial}} = \theta^{(m)} + \delta^{(m)}$ ;
7      计算试探参数对应的目标函数值;
8      if 目标函数下降 then
9          接受该步, 令  $\theta^{(m+1)} = \theta_{\text{trial}}$ ;
10         适当减小阻尼参数  $\lambda^{(m)}$ ;
11          $m \leftarrow m + 1$ ;
12     else
13         拒绝该步并增大阻尼参数, 重新计算当前迭代的试探步;
14     end
15 end
16 从最终参数向量中拆分出  $y_0$  与  $\alpha$ ;
17 返回  $y_0$  与  $\alpha$ ;

```

第四节 测试误差评估流程

算法 B.4 测试误差评估流程

Data: 训练得到的 y_0, α

Result: 独立测试样本上的终端残差 RMSE

- 1 重新生成独立测试路径 $\{X_k^{(i)}, \Delta W_k^{(i)}\}$;
- 2 在测试路径上构造随机特征 $\{H_k^{(i)}\}$;
- 3 用固定参数 α 构造 $Z_k^{(i)} = \alpha H_k^{(i)}$;
- 4 沿测试路径前向推进离散 BSDE, 得到所有终端值 $Y_N^{(i)}$;
- 5 计算

$$\text{RMSE}_{\text{test}} = \sqrt{\frac{1}{S} \sum_{i=1}^S (Y_N^{(i)} - g(X_N^{(i)}))^2};$$

返回测试 RMSE;

附录 C 参考值的计算与可靠性说明

本附录说明第三章中各算例参考值的计算方法. 对具有显式解的方程, 直接使用解析公式. 对没有简单闭式解的方程, 使用与正文 RFM 求解器无关的数值方法计算参考值, 并给出相应的置信区间或误差估计.

第一节 Heat 方程的显式参考值

Heat 方程的终端条件取为

$$g(x) = \exp\left(-\frac{\|x\|^2}{d}\right), \quad x_0 = 0, \quad T = 1. \quad (\text{C.1})$$

更一般地, 若终端条件写为

$$g(x) = \exp\left(-\frac{\|x\|^2}{a}\right),$$

则由 Feynman–Kac 公式可得

$$u(t, x) = \left(\frac{a}{a + 2(T - t)}\right)^{d/2} \exp\left(-\frac{\|x\|^2}{a + 2(T - t)}\right). \quad (\text{C.2})$$

因而在 $x_0 = 0, a = d$ 时,

$$Y_0 = \left(\frac{d}{d + 2}\right)^{d/2}. \quad (\text{C.3})$$

代入不同维度得到表 C.1. 这些值由显式公式直接计算, 不包含额外的数值离散误差.

表 C.1 Heat 方程参考值

d	Y_0
20	0.385543289429532
30	0.379812405815246
50	0.375116802253964
100	0.371527882126961
200	0.369711212329119
300	0.369102310996184

第二节 HJB-LQ 方程的积分参考值

HJB-LQ 方程的终端条件取为

$$g(x) = \log\left(\frac{1 + \|x\|^2}{2}\right). \quad (\text{C.4})$$

HJB-LQ 方程对应的 BSDE 为

$$dY_t = \frac{1}{2} \|Z_t\|^2 dt + Z_t^T dW_t. \quad (\text{C.5})$$

令 $U_t = \exp(-Y_t)$, 则 U_t 为鞅. 于是

$$Y_0 = -\log \mathbb{E} \left[\frac{2}{1 + 2T\chi_d^2} \right]. \quad (\text{C.6})$$

因此参考值可以通过下面的一维积分计算:

$$\mathbb{E} \left[\frac{2}{1 + 2T\chi_d^2} \right] = \int_0^\infty \frac{2}{1 + 2Ty} f_{\chi_d^2}(y) dy. \quad (\text{C.7})$$

本文采用自适应 Simpson 积分在有限区间上计算该积分. 当积分上限从 300 增加到 500 时, 结果变化不超过 5×10^{-12} , 因而在本文所列精度下, 截断误差可以忽略. 最终结果见表 C.2.

表 C.2 HJB-LQ 方程参考值

d	Y_0
20	2.921014735738689
30	3.351223237182463
50	3.882006682195226
100	4.590161724604434
200	5.290814736008237
300	5.698781231260821

第三节 Black-Scholes 方程的 Monte Carlo 参考值

对 Black-Scholes 方程, 终端收益函数为高维算术平均型欧式看涨期权

$$g(x) = \max \left(\frac{1}{d} \sum_{m=1}^d x_m - K, 0 \right). \quad (\text{C.8})$$

该问题没有类似一维 Black-Scholes 公式的简单闭式解. 因此本文采用独立的大样本 Monte Carlo 方法估计其初值, 并使用几何平均期权的闭式定价公式作为控制变量.

记算术平均收益对应的贴现随机变量为 A , 几何平均收益对应的贴现随机变量为 G . 由于几何平均

$$\bar{G}_T = \exp \left(\frac{1}{d} \sum_{m=1}^d \log X_T^m \right) \quad (\text{C.9})$$

仍服从对数正态分布, 因而 $\mathbb{E}[G]$ 可以显式计算. 于是采用控制变量估计量

$$A^* = A - \beta(G - \mathbb{E}[G]), \quad (\text{C.10})$$

同时结合对偶路径进一步降低方差.

本文使用 10^6 条路径计算参考值, 结果和 95% 置信区间见表 C.3.

表 C.3 Black–Scholes 方程 Monte Carlo 参考值

d	Y_0	95% 置信区间
20	0.05173661	[0.05172011, 0.05175310]
30	0.05021835	[0.05020434, 0.05023235]
50	0.04921751	[0.04920643, 0.04922860]
100	0.04881270	[0.04880488, 0.04882053]
200	0.04877301	[0.04876747, 0.04877854]
300	0.04876993	[0.04876541, 0.04877445]

第四节 Allen–Cahn 方程的有限差分参考值

对 Allen–Cahn 方程, 由于其终端条件和扩散过程都具有径向对称性, 该问题可以转化为径向变量上的一维半线性抛物方程. 记 $r = \|x\|$, 则解可写为 $u(t, x) = v(t, r)$. 在反向时间变量 $\tau = T - t$ 下, 参考解满足

$$\frac{\partial v}{\partial \tau} = \frac{\partial^2 v}{\partial r^2} + \frac{d-1}{r} \frac{\partial v}{\partial r} + v - v^3, \quad v(0, r) = \frac{1}{2 + 0.4r^2}. \quad (\text{C.11})$$

本文在区间 $r \in [0, 20]$ 上采用径向有限差分离散, 对扩散项作隐式处理, 对非线性反应项作显式处理. 在 $r = 0$ 处施加径向对称边界条件, 在远端边界取零值近似.

为评估离散误差, 分别使用两组网格

$$(\Delta r, \Delta \tau) = (0.025, 5 \times 10^{-5}),$$

$$(\Delta r, \Delta \tau) = (0.0125, 2.5 \times 10^{-5})$$

进行计算, 并将两组结果的差作为误差估计. 最终采用细网格结果作为参考值, 结果见表 C.4.

此外, 当边界截断半径从 15 增加到 30 时结果几乎不变, 因而边界截断误差小于当前网格误差.

表 C.4 Allen–Cahn 方程有限差分参考值

d	Y_0	网格差估计
20	0.20635880	2.08×10^{-6}
30	0.15199385	1.59×10^{-6}
50	0.09908593	3.19×10^{-6}
100	0.05278464	2.81×10^{-6}
200	0.02724542	1.79×10^{-6}
300	0.01835709	1.29×10^{-6}

致 谢

在研究学习期间，我有幸得到了三位老师的教导，他们是：我的导师，中国科大 XXX 研究员，中科院 X 昆明动物所马老师以及美国犹他大学的 XXX 老师。三位深厚的学术功底，严谨的工作态度和敏锐的科学洞察力使我受益良多。衷心感谢他们多年来给予我的悉心教导和热情帮助。

感谢 XXX 老师在实验方面的指导以及教授的帮助。科大的 XXX 同学和 XXX 同学参与了部分试验工作，在此深表谢意。

2026 年 5 月